ECE 496C / 519C: Blockchain Technologies
Assignment 1

Weighting: 100 Marks Total
Deadline: October 22, 2026, at 11:59 PM (16 days)
Submission: Single `.zip` archive on Brightspace
Format: Individual Assignment (Each student runs a 3-node network locally on their machine)

Overview

In this assignment, you will build a lightweight Layer 1 Blockchain featuring a custom Legacy Bitcoin Vanity Address generator, an interactive Proof-of-Work (PoW) mining module, and a Peer-to-Peer (P2P) network. This is an individual assignment. You are free to use any programming language (e.g., Python, JavaScript/TypeScript, Go, Rust, C++). All screenshots and examples provided in this handout are for illustrative purposes only.

Part 1: Legacy Bitcoin Vanity Address Generator (20 Marks)

Before your nodes can mine or communicate, you must generate a valid Bitcoin Legacy address containing a custom vanity prefix.

Address Generation Logic

Your identity generator must implement standard Bitcoin Legacy (P2PKH) address generation:

1. Generate an ECDSA secp256k1 keypair and extract the compressed public key (33 bytes).
2. Hash the public key: RIPEMD160 (SHA256 (PublicKey)).
3. Prepend the mainnet network version byte (`0x00`) to the public key hash.
4. Calculate 4-byte checksum using double SHA-256: `SHA256(SHA256(VersionedHash))[:4]`.
5. Append the 4-byte checksum to the versioned hash and encode the final byte sequence using **Base58**.

Vanity Matching Requirement

- All Legacy Bitcoin addresses begin with the character `'1'`.
- **Characters 2, 3, and 4** of your generated address must match the **first three letters of your first name**, ignoring upper/lowercase differences.
- Prefix Extraction Rule: Use the first three letters (A–Z) of your first name. If your first name is shorter than 3 letters, continue with your last name. Ignore accents, hyphens, and apostrophes.
- *Example:* For student "Justin", the required prefix is `"JUS"`. A valid matching address is `1JuSFZ4T582XUQUKsP47M5fEPeYTQ8mP1z` (where characters 2–4 are `"JuS"`).

Why Case-Insensitive? Base58 encoding excludes characters `0`, `O`, `I`, and `l` to prevent visual ambiguity. Ignoring cases ensure that every student's name combination remains mathematically achievable.

Important Notes for Students

1. **Search Speed:** Names starting with letters A through P typically find a match in under 60 seconds. Names starting with I, O, or R through Z are rarer in Base58 character positions and may take several minutes. **In that case, you should save your generated vanity keypair to a file (`keypair.json`) and reuse it.** Do not search for a new vanity address during your live demo.
2. **Python Hashlib Note:** Some default Python environments do not enable RIPEMD-160 in `hashlib`. Installing `pycryptodome` (`pip install pycryptodome`) provides full RIPEMD-160 support (`Crypto.Hash.RIPEMD160`).
3. keypair.json is strictly for this assignment and must never be used for real funds.

Execution Example (Part 1)

```
Node1> python3 main.py --config node1.json --vanity-prefix JUS --difficulty 4

[+] Loading keypair from keypair.json... (Found existing match)
[+] Public Address : 1JuSFZ4T582XUQUKsP47M5fEPeYTQ8mP1z
[+] Prefix Status  : MATCH FOUND ("JuS" matches "JUS" case-insensitive)
[+] Node Identity initialized successfully.
```

Part 2: Genesis Block, Data Structure & Interactive Mining (40 Marks)

Hardcoded Genesis Block

To ensure all independent nodes share a common state, your application must hardcode an identical Genesis Block across all nodes:

- **Index:** `0`
- **Previous Hash:** `"0000000000000000000000000000000000000000000000000000000000000000"`
- **Timestamp:** `"2026-01-01T00:00:00Z"`
- **Nonce:** `0`
- **Miner Address:** `"GENESIS"`
- **Data:** `["Genesis Block"]`

Note: The Genesis block is hardcoded and trusted. It is not checked for Proof-of-Work, and its hash is not required to meet the difficulty target.

Block Structure & Hashing

Each block in your ledger contains:

1) index
2) prev_hash
3) timestamp
4) data (an array of text strings)
5) nonce
6) miner_address (the miner's Part 1 vanity address)
7) hash

Critical Hashing Rule: The `hash` field itself is **NOT** part of the hash input calculation. You must serialize `index + timestamp + prev_hash + data + nonce + miner_address` into a deterministic string before computing the SHA-256 hash. Document your exact field order and formatting in your `README.md`.

Mining on Command

Instead of continuous background mining, your node must mine **on demand**:

- Typing `mine` in a node's terminal (or invoking its `/mine` HTTP endpoint) triggers that node to collect pending text messages, insert its vanity address into `miner_address`, find a valid PoW nonce, broadcast the block, and stop.
- **Difficulty:** Default difficulty target is **4 leading hex zeros** (`"0000..."`), adjustable dynamically via the `--difficulty` flag.
- Pending Messages: Implement a `msg <text>` command (or a `/message` HTTP endpoint) to allow adding text messages. Messages remain local to the node. If no pending messages exist when mining is triggered, the block is mined with an empty data list [].
- Local Chain Update: Upon successfully mining a block, the node adds the block to its own local chain before broadcasting it to peers.

Execution Example (Part 2)

```
Node1> mine

[+] Triggering Mining Cycle on Node1...
[+] Block #1 Target Difficulty: 4 leading zeros ("0000...")
[+] Nonce: 0      -> Hash:
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
[+] Nonce: 1842   -> Hash:
0000a8f12c9b4e5d6f7a8b9c0d1e2f3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d9e
[+] Block #1 Mined! Nonce: 1842
[+] Block Hash: 0000a8f12c9b4e5d6f7a8b9c0d1e2f3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d9e
[+] Miner: 1JuSFZ4T582XUQUKsP47M5fEPeYTQ8mP1z
```

Part 3: P2P Communication, Block Validation & Consensus (40 Marks)

P2P Communication Protocol

You may implement peer communication using **simple HTTP endpoints** (e.g., Flask/FastAPI in Python, Express in Node.js) or raw **TCP sockets**.

- Network nodes read peer definitions from individual configuration files (`node1.json`, `node2.json`, `node3.json`).
- If a configured peer is offline, the broadcasting node must log a warning and continue operating without crashing.
- Direct Connections: Since all three nodes are directly connected, nodes do not need to re-forward or gossip received blocks to other peers.
- Memory State: Saving the blockchain to disk is not required. All three nodes should be started before mining begins.

Block Validation Checklist

When Node B receives a mined block from Node A, Node B must perform the following four checks before appending it to its local chain:

1. **Hash Verification:** Recompute the hash from the block payload fields (excluding the `hash` field) and confirm it exactly equals the block's `hash` attribute.
2. **Difficulty Verification:** Check that the recomputed hash begins with the required number of leading zeros (e.g., `"0000"`).
3. **Chain Continuity:** Confirm that the block's `prev_hash` equals the hash of Node B's current tip block.
4. **Index Sequence:** Confirm that the block's `index` equals Node B's current tip `index +1`.

Consensus & Fork Limitation

For this assignment, implement **First-Valid-Block-Wins** consensus: nodes accept the first valid block received that extends their current height and reject conflicting blocks at the same height.
Known System Limitation: If two nodes mine a block at the same height simultaneously, a network split (fork) will occur because neither node will yield to the other. You are **not** required to fix this in Assignment 1 (handling forks using the most-work chain rule will be introduced in Assignment 2). However, you will be asked to demonstrate this limitation during your viva.

Tip for Demo: Use `--difficulty 6` during the fork demo, so mining takes enough time to trigger simultaneously on two nodes.

Standard Command Format

Your application must use uniform CLI parameters across all nodes:

```
Node1> python3 main.py --config node1.json --vanity-prefix JUS --difficulty 4
```

Sample Configuration (`node1.json`):

```
{
  "node_id": "Node1",
  "port": 8001,
  "peers": ["http://127.0.0.1:8002", "http://127.0.0.1:8003"]
}
```

Submission & README Requirements

Your submission `.zip` file must include:

1. **Source Code:** Full implementation.
2. **Configurations:** `node1.json`, `node2.json`, and `node3.json`.
3. **Keypair File:** `keypair.json` containing your pre-computed vanity keypair.
4. **`README.md`:** Must contain:
    a. Exact setup and run commands for your 3 nodes.
    b. The explicit field order and formatting string used to hash blocks.
5. **`AI_Usage_Log.md`:** Standard documentation of prompts, outputs, and modifications if Generative AI was used. **If no AI tools were used, you should still submit this file stating no use of AI.**

Demonstration & Oral Defense (15 Minutes)

Marks for Part 1 (20 marks), Part 2 (40 marks), and Part 3 (40 marks) are evaluated and awarded based on your live demo performance and code viva. Demonstrations will be conducted in-person or online. An Excel Sheet containing demo slots will be announced on Brightspace. Students must book their 15-minute evaluation slot. More details on this will be posted on Brightspace.

Live Demo Evaluation Checklist

During your 15-minute slot, you will:

1. **Launch 3 Nodes:** Start 3 local instances using `node1.json`, `node2.json`, and `node3.json` loaded with your pre-computed vanity key (`1JuSF...`).
2. **On-Demand Mining:** Call `mine` on Node 1, showing Node 2 and Node 3 validating and appending Block #1.
3. **Fork Triggering**: Run nodes with --difficulty 6 and simultaneously trigger mine on both Node 2 and Node 3 at the same block height to produce a chain split, demonstrating the "First-Valid-Block-Wins" limitation.
4. **Code Viva:** Answer technical questions regarding your hashing, HTTP/socket routing, and validation logic.